PROJECT PHASE I | REVIEW 1

CareerCompass AI

Live Job Market Analytics, Career Recommendation, Skill-Gap Analysis and Personalized Learning Roadmap System

SYNOPSIS

A proposed, explainable data-driven system that converts permitted job-posting data into market insights and personalized career-learning guidance.

STUDENT DETAILS	TO BE COMPLETED BEFORE SUBMISSION
Student name / roll number	[Enter student details]
Department / institute	[Enter department and institute]
Project guide	[Enter guide name]
Academic year / semester	[Enter academic year and semester]

Synopsis status

Prepared against the five Review 1 points supplied by the student. No official evaluation-weightage breakdown was included; any rubric later in this document is explicitly illustrative.

Submission note: replace bracketed student/institution fields, confirm local formatting rules, validate API access and terms for the target country, and review the proposed effort and costs with the project guide.

1. Problem statement, literature basis, motivation and objectives

Proposed problem statement. Students lack a single, transparent workflow for relating their self-reported skills and career interests to observable job-market requirements and then turning skill gaps into actionable learning steps. Job-posting information is distributed across listings, descriptions may be incomplete, and raw market counts can be difficult to interpret. This project will build a web system that collects permitted job listings through an authorized API, cleans and analyzes them, explains skill-based role matches, and generates a learning roadmap.

Proposed title. CareerCompass AI: Live Job Market Analytics, Explainable Career Recommendation, Skill-Gap Analysis and Personalized Learning Roadmap System. ?Live? here means API-connected, periodically refreshed data; the implementation must not imply continuous real-time monitoring unless it actually provides it.

Preliminary literature and technology review

The initial literature direction is recommender systems and content-based recommendation. Established recommender-system literature describes recommendation as matching user preferences with item characteristics; content-based approaches are appropriate to investigate when recommendations should be explained through explicit features. For this project, student-declared skills and role-skill requirements are the transparent features. The proposed contribution is an integrated student-facing workflow, not a claim that job recommendation itself is novel.

SOURCE	RELEVANCE TO THIS PROJECT
Adomavicius & Tuzhilin (2005), ?Toward the Next Generation of Recommender Systems,? IEEE TKDE 17(6), DOI: 10.1109/TKDE.2005.99.	Background on recommender-system approaches and design considerations.
Pazzani & Billsus (2007), ?Content-Based Recommendation Systems,? The Adaptive Web, DOI: 10.1007/978-3-540-72079-9_10.	Background for describing item features and explainable content-based matching.
Adzuna API documentation: developer.adzuna.com/overview and /docs/search.	Documents API registration credentials and job-search fields; access/quotas/terms need confirmation.
FastAPI, pandas, scikit-learn official documentation.	Implementation references for API validation, analysis, and similarity methods.

Literature gap and feasibility rationale

The project will evaluate whether a practical prototype can combine (1) traceable job-market summaries, (2) explainable skill-based role matching, and (3) an actionable learning roadmap in one system. This is a project scope to investigate, not a claim that existing products do not offer these features. A formal literature survey should be expanded with institution-approved scholarly databases before final submission.

Motivation

Provide students a data-informed starting point for career exploration while making uncertainty visible. The project also offers practical learning in API integration, data engineering, analytics, full-stack development, database design, and responsible use of AI.

Objectives

- Collect job listings from an authorized API with pagination, error handling, source attribution, and refresh timestamps.
- Clean and deduplicate jobs; extract normalized skills with a transparent, editable dictionary.
- Calculate job-market indicators, especially demand as the share of unique selected postings mentioning each skill.
- Recommend career roles using explainable content-based matching and show matching/missing skills.
- Prioritize skill gaps and prepare weekly learning steps using verified resources.
- Build a tested FastAPI + React application backed by MySQL and communicate data limitations.

2. Scope and feasibility study

Project scope

IN SCOPE	OUT OF SCOPE / LIMITATION
One permitted job API integration; configurable country/query; periodic refresh; persistent raw/clean data.	Unauthorized website scraping; unlimited or guaranteed API access; continuous real-time crawling.
Deduplication; missing-value handling; editable skill dictionary; quality report.	Perfect extraction from incomplete descriptions; claims that every listed skill is a job requirement.
Filtered analytics for unique listings, skill demand, titles/locations, optional salary/experience and valid trends.	Salary comparison across currencies without normalization; historical trends without dated/snapshot data.
Student-entered profile, explainable role similarity, gap labels, curated learning roadmap.	Hiring probability, guaranteed employment outcome, automated psychological/personality assessment.
Responsive web interface and REST APIs; minimal student data.	Production-scale multi-tenant service or sensitive personal-data collection in this academic prototype.

Feasibility

DIMENSION	ASSESSMENT AND CONDITIONS
Technical	Feasible as a modular prototype using Python, FastAPI, MySQL, React, and documented libraries. Depends on a registered provider account, network connectivity, and data fields returned.
Economic	Can be developed locally with free/open-source tools. Incremental costs mainly depend on internet, optional hosting, and any provider plan; verify actual current charges before commitment.
Operational	A student can run periodic refreshes and review quality reports. Users need clear source dates, honest empty/error states, and a way to correct self-reported skills.
Schedule	A staged semester plan is plausible if API access and database setup are confirmed early. Prioritize collection, cleaning, and analytics before advanced recommendation work.
Legal / ethical	Use only permitted API access and comply with provider terms. Minimize student data; protect credentials; disclose sample bias and uncertainty; do not present similarity as hiring probability.

Risks and mitigations

API access or quota constraints: verify access first, cap pages, cache results, and provide documented test fixtures. Partial/noisy job descriptions: disclose coverage, use editable matching rules, and test false positives. Sparse salary/date fields: hide or qualify unsupported analysis. Personal data risk: use minimal profile fields and do not store profiles unless necessary.

Feasibility decision

Proceed as a local academic prototype after confirming API registration, target-country coverage, provider terms, and departmental expectations for data retention.

3. Resources, test environment, cost and effort

Hardware and other resources

RESOURCE	MINIMUM / PLANNING SPECIFICATION	COST BASIS
Development computer	Existing Windows 10/11 laptop/desktop; recommended 8 GB RAM, dual-core 64-bit CPU, 10 GB free disk.	Assumed available; no purchase included.
Network	Internet for package installation and API refresh; browser for testing.	Existing connection; monthly cost varies.
Optional demo display	Projector / presentation display for review.	Use institute equipment if available.

Software and test tools

AREA	PROPOSED TOOLING
Backend / analysis	Python 3.11+; FastAPI; Uvicorn; HTTPX; Pydantic Settings; SQLAlchemy; PyMySQL; Pandas; NumPy; scikit-learn; pytest.
Database	MySQL Community Server; MySQL Workbench optional.
Frontend / charts	Node.js LTS + npm; React; Vite; React Router; Plotly / Plotly.js; responsive CSS.
Development / verification	VS Code; Git/GitHub; Swagger UI (/docs); browser DevTools; pytest; API mocks/test fixtures; MySQL client.
Environment	Local Windows development; backend and frontend run as separate local processes; local MySQL. Optional cloud deployment is not required for initial review.

Planning cost estimate (INR)

Indicative, not a vendor quote. Assumes an existing computer, local development, and no paid hosting. Confirm current prices and account terms locally.

ITEM	PLANNING ASSUMPTION	ESTIMATE
Python, React, FastAPI, MySQL Community, Git, VS Code	Open-source/free developer tools; verify each license and current terms.	INR 0 software license budget
Computer	Already owned/available.	INR 0 incremental
Internet	Existing plan; illustrative allowance only.	INR 500-INR 1,500/month (local plan dependent)
Job API	Adzuna developer account required; quotas and terms depend on current account.	Unpriced until account terms checked
Hosting / domain	Optional; use local demo for Phase I.	INR 0 for local prototype; optional cost TBD

Software measurement plan

Measure delivered size using a requirements-based Function Point (FP) count after endpoints, external interfaces, logical data groups, and user transactions are agreed; retain the count sheet and assumptions. Also record actual source lines of code (SLOC) by language from the final repository, excluding generated/vendor files. Do not present planned feature counts as measured size. Proposed functional inventory to validate: six user-facing pages, approximately eight REST endpoint groups, and the required relational entities plus join tables.

Human effort estimate (planning estimate)

WORK PACKAGE	HOURS
Requirements, feasibility, literature, and design	18
Environment, API access, and backend foundation	16
Collection, cleaning, and skill extraction	28
MySQL schema and persistence	16
Analytics APIs and calculations	20
React pages, charts, and API integration	26
Recommendation, gaps, and roadmap	24
Testing, fixes, documentation, demo preparation	20
TOTAL PLANNING EFFORT	168 hours

Effort is an initial student-person-hour estimate, not a measured result or guarantee. Track actual hours by task and update the estimate at each review.

Planning notes: cost, size, and effort

Cost assumptions

INR estimates are illustrative planning values, not quotes. The local prototype assumes an existing computer and free/open-source development tools. Internet cost varies by provider and plan. API access, quotas, and any hosting charge must be checked against current terms before spending.

Software size measurement

After requirements are agreed, count Function Points using documented inputs, outputs, inquiries, logical data groups, and external interfaces. At project completion, record source lines of code by language, excluding generated and vendor files. Report measurement date and counting assumptions.

Effort tracking

The 168 student-person-hours are a planning estimate. Keep a simple task log with planned hours, actual hours, and completion status; revise estimates at each review. The estimate is not a measured result or guarantee.

Cost to finalize

Confirm whether the department expects capital cost, recurring cost, or total project cost. Add locally verified costs for any required equipment, internet, API plan, hosting, or presentation materials; otherwise label them as unavailable/not applicable.

4. Proposed system overview and expected outcomes

Proposed workflow

STAGE	SYSTEM ACTION	OUTPUT
1. Collect	FastAPI service calls an authorized job-search API using server-side credentials; applies pagination, timeout, and error handling.	Raw/source listings and collection metadata.
2. Prepare	Normalize fields, deduplicate, detect missing values, extract dictionary-based skills.	Clean jobs, job-skill links, quality report.
3. Analyze	Calculate unique-posting counts and skill demand; aggregate location/title and supported salary/date indicators.	Filtered analytics data with denominator and source window.
4. Compare	Normalize student-entered skills and compare against role-skill requirements and optional preference filters.	Ranked role matches with score explanation and skill overlaps.
5. Plan	Prioritize role gaps and attach verified resources, weekly targets, practice, and progress status.	Personalized learning roadmap.
6. Present	React UI calls backend JSON endpoints and visualizes status, analytics, recommendations, and learning steps.	Responsive dashboard and student-facing guidance.

Expected outcomes / deliverables

- Working source-connected data collection module, with credentials held outside version control.
- Separate raw and cleaned datasets plus documented quality summary.
- MySQL schema for jobs, skills, student profiles, roles, resources, and roadmap information.
- Dashboard that discloses data source, sample size, filters, date range, and last successful refresh.
- Explainable recommendations, categorized skill gaps, and editable learning milestones.
- Automated tests, README, configuration example, and repeatable local demo instructions.

Success criteria (functional, to be verified)

A reviewer can run the application locally, observe a health response, refresh or load permitted/test data, inspect quality metrics, filter analytics, submit a profile, understand the role-match explanation, and view a roadmap. Numerical performance or model-accuracy claims will be reported only after measurement on a documented test set.

Data interpretation

Skill demand = 100 ? (number of unique selected job postings mentioning the skill ? number of unique selected job postings). Always display the denominator and applicable source/date window.

5. Architecture and initial design (DFD)

Proposed logical architecture

External entity: Student
Enters filters/profile; views results

React + Vite interface
Pages, validation, loading/error states, Plotly views

HTTPS / JSON REST requests

FastAPI backend
API routes + schemas | collection | cleaning/analytics | recommendation/roadmap services

Authorized provider API calls (server-side secrets) + SQL persistence

MySQL
Jobs / skills / links / roles / minimal student skills / resources / roadmap

DFD Level 0 (context diagram)

Student Profile, filters, requests	**CareerCompass AI** Career analytics and guidance system	**Job Data API** Authorized listing responses
	? API request / search parameters ? ? listings, source fields, API errors ?	
? Dashboard, recommendations, gaps, roadmap ?		? Search query / credentials held server-side ?

DFD Level 1 (major processes and data stores)

PROCESS	INPUT ? PROCESS ? OUTPUT	DATA STORE
1.0 Collect jobs	Search config ? authorized API client ? raw jobs + refresh result	D1 Raw/Jobs
2.0 Clean and extract	Raw jobs ? normalize/deduplicate/skill match ? clean jobs + quality report	D1, D2 Skills
3.0 Analyze market	Filters + clean jobs ? aggregates ? charts/summary	D3 Analytics (derived; can be queried)
4.0 Recommend / gap	Profile + role skills + demand ? explainable score/gap priorities	D4 Roles/Profile
5.0 Build roadmap	Selected role + prioritized gaps ? milestones/resources/progress	D5 Roadmaps/Resources

Primary data stores

D1 Jobs (raw/source and cleaned state or separate tables); D2 Skills and JobSkills; D3 derived analytics or refresh snapshots; D4 CareerRoles, role-skill requirements, and minimal student profile/skills; D5 LearningResources, LearningRoadmaps, and milestones. The logical schema should enforce foreign keys and prevent duplicate job-skill links.

Key data-flow and trust decisions

- API credentials remain only in backend environment configuration; never flow to the browser.
- Job source URL/provider, posted_date, collected_at, refresh status, and sample window travel with analytics metadata.
- Only minimal student information is accepted/stored; profile fields are validated and are not used as protected-attribute scoring features.
- Recommendation response includes score method, matched/missing skills, and limitations.

6. Milestones, evaluation format and references

Initial development milestones

MILESTONE	OUTPUT / REVIEW EVIDENCE
A. Feasibility + design	Confirmed provider access plan, requirements, ER model, DFD, environment setup.
B. Data foundation	Mocked API tests, successful permitted test retrieval, raw/clean records, quality report.
C. Analytics	Verified deduplication and demand formula; filterable backend responses.
D. Web integration	React dashboard and status/error states using backend endpoints.
E. Guidance features	Recommendation explanation, skill-gap summary, roadmap resources/progress.
F. Verification + presentation	Automated tests, user walkthrough, documented limitations, final measured effort/size.

Review 1 evaluation criteria and weightage

Rubric note

The quoted message refers to a suggested evaluation table but did not include its criteria or weights. The following is an illustrative 100-point proposal only. Replace it with the department/guide?s official rubric before submission.

PROPOSED CATEGORY (ILLUSTRATIVE)	WEIGHT
Problem definition, preliminary literature review, and feasibility	20
Motivation, objectives, scope, and requirements clarity	15
System overview, architecture, and DFD design	25
Technology/resources, cost, measurement, and effort planning	15
Methodology, milestones, risks, and testing plan	15
Presentation, report quality, and viva responses	10
TOTAL (proposed only)	100

References (initial; expand literature survey before final submission)

[1] G. Adomavicius and A. Tuzhilin, ?Toward the Next Generation of Recommender Systems: A Survey of the State-of-the-Art and Possible Extensions,? IEEE Transactions on Knowledge and Data Engineering, vol. 17, no. 6, pp. 734?749, 2005. DOI: 10.1109/TKDE.2005.99.
[2] M. J. Pazzani and D. Billsus, ?Content-Based Recommendation Systems,? in The Adaptive Web, 2007, pp. 325?341. DOI: 10.1007/978-3-540-72079-9_10.
[3] Adzuna Developer API, API overview and job search documentation (accessed 12 Oct 2026; confirm current terms/account availability).
[4] FastAPI, Tutorial; pandas, Getting started; scikit-learn, Pairwise metrics; React, Learn; Plotly, Python documentation.

Review 1 closing statement

The proposed work is a feasible staged prototype: begin with authorized data collection and measurable analytics, then add explainable student-to-role matching and a verifiable learning roadmap.